CSA1737 – ARTIFICIAL INTELLIGENCE

LIST OF EXPERIMENTS

1.Write the Python program to solve 8-Queen problem.

AIM:

To write a Python program to solve the 8-Queen problem using backtracking.

ALGORITHM:

1. Start the program.
2. Create an 8 × 8 chessboard.
3. Place one queen in each row.
4. Check whether the queen conflicts with other queens.
5. If safe, place the queen and move to the next row.
6. If no position is safe, backtrack.
7. Display the solution.
8. Stop the program.

CODE:

main.py

```
N = 8
board = [-1] * N
def is_safe(row, col):
    for i in range(row):
        if board[i] == col:
            return False
        if abs(board[i] - col) == abs(i - row):
            return False
    return True
def solve(row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False
if solve(0):
    for row in range(N):
        for col in range(N):
```

```
    return False
if solve(0):
    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution exists")
```

OUTPUT:

Output

```
Q - - - - - - -
- - - - Q - - -
- - - - - - - Q
- - - - - Q - -
- - Q - - - - -
- - - - - - Q -
- Q - - - - - -
- - - Q - - - -
```

12.Write the Python program for Water Jug Problem.

AIM:

To write a Python program to solve the Water Jug Problem using a state-space search.

ALGORITHM:

1. Start the program.
2. Initialize two jugs of capacities 4 litres and 3 litres.
3. Set the initial state as (0, 0).
4. Generate possible states by filling, emptying and transferring water between jugs.
5. Avoid revisiting states.
6. Stop when the 4-litre jug contains exactly 2 litres.
7. Display the solution path.
8. Stop the program.

CODE :

main.py

```python
from collections import deque
def water_jug():
    capacity_a = 4
    capacity_b = 3
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (a, b), path = queue.popleft()
        if a == 2:
            return path
        transfer_ab = min(a, capacity_b - b)
        transfer_ba = min(b, capacity_a - a)
        states = [
            (capacity_a, b),
            (a, capacity_b),
            (0, b),
            (a, 0),
            (a - transfer_ab, b + transfer_ab),
            (a + transfer_ba, b - transfer_ba)
        ]
        for state in states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None
solution = water_jug()
if solution:
    for state in solution:
        print(state)
else:
    print("No solution exists")
```

OUTPUT:

Output

```
(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
```

13. Write a Python program for Crypt-Arithmetic problem

AIM:

To write a Python program to solve a crypt-arithmetic problem in which each letter represents a unique digit.

ALGORITHM:

1. Start the program.
2. Define the words in the arithmetic equation.
3. Assign different digits to the letters.
4. Ensure that leading letters are not zero.
5. Check whether the arithmetic equation is correct.
6. Display the valid solution.
7. Stop the program.

CODE :

main.py

```
from itertools import permutations
letters = "SENDMORY"
digits = "0123456789"
for p in permutations(digits, len(letters)):
    d = dict(zip(letters, p))
    if d["S"] == "0" or d["M"] == "0":
        continue
    send = int("".join(d[x] for x in "SEND"))
    more = int("".join(d[x] for x in "MORE"))
    money = int("".join(d[x] for x in "MONEY"))
    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break

```

OUTPUT:

Output

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

14. Write a Python program for Missionaries Cannibal problem.

AIM:

To write a Python program to solve the Missionaries and Cannibals problem using Breadth-First Search (BFS).

ALGORITHM:

1. Start the program.
2. Initialize three missionaries and three cannibals on the left bank.
3. Use a boat that can carry one or two people.
4. Generate possible boat crossings.
5. Check that missionaries are never outnumbered by cannibals on either bank when missionaries are present.
6. Use BFS to find the solution.
7. Display the sequence of crossings.
8. Stop the program.

CODE:

main.py

```
from collections import deque
start = (3, 3, 1)
goal = (0, 0, 0)
moves = [(1, 0), (2, 0), (0, 1),
         (0, 2), (1, 1)]
def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and m < c:
        return False
    mr = 3 - m
    cr = 3 - c
    if mr > 0 and mr < cr:
        return False
    return True
def bfs():
    queue = deque([(start, [])])
    visited = {start}
    while queue:
        state, path = queue.popleft()
        m, c, boat = state
        if state == goal:
        for dm, dc in moves:
            if boat == 1:
                new_state = (m - dm, c - dc, 0)
            else:
                new_state = (m + dm, c + dc, 1)
            if valid(new_state[0], new_state[1]):
                if new_state not in visited:
                    visited.add(new_state)
                    queue.append(
                        (new_state, path + [state])
                    )
    return None
solution = bfs()
if solution:
    for state in solution:
        print(state)
else:
    print("No solution exists")
```

OUTPUT:

Output

```
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

15. Write a Python program for Vacuum Cleaner problem.

AIM:

To write a Python program to implement the Vacuum Cleaner problem using a simple reflex agent.

ALGORITHM:

1. Start the program.
2. Initialize the status of two rooms, A and B.
3. Place the vacuum cleaner in room A.
4. Check whether the current room is dirty.
5. If dirty, clean the room.
6. Move to the other room and repeat.
7. Display the final room status.
8. Stop the program.

CODE:

main.py

```
rooms = {
    "A": "Dirty",
    "B": "Dirty"
}
location = "A"
while True:
    print("Vacuum is in room", location)
    if rooms[location] == "Dirty":
        rooms[location] = "Clean"
        print("Room", location, "cleaned")
    if all(status == "Clean" for status in rooms.values()):
        break
    if location == "A":
        location = "B"
    else:
        location = "A"
print("Final room status:", rooms)
```

OUTPUT:

Output

```
Vacuum is in room A
Room A cleaned
Vacuum is in room B
Room B cleaned
Final room status: {'A': 'Clean', 'B': 'Clean'}
```